

RESEARCH ARTICLE

WILEY

LightCert: On designing a lighter certificate for resource-limited Internet-of-Things devices

Hyuksang Kwon¹ | Jeongseob Ahn¹ | JeongGil Ko² 

¹Department of Computer Engineering,
Ajou University, Suwon, South Korea

²School of Integrated Technology, Yonsei
University, Seoul, South Korea

Correspondence

JeongGil Ko, School of Integrated
Technology, Yonsei University,
85 Songdogwahakro, Yeonsugu,
Incheon 21983, South Korea.
Email: jeonggil.ko@yonsei.ac.kr

Funding information

Korea Institute for Advancement of
Technology, Grant/Award Number:
International Cooperative R&D program;
National Research Foundation of Korea,
Grant/Award Number:
2018R1C1B6003869

Abstract

Internet-of-Things (IoT) devices are typically resource constrained in terms of computing capabilities and battery power. Despite the efforts from the Internet Engineering Task Force (IETF) to established standards for IoT such as IPv6 over low-power wireless personal area networks (6LoWPAN), routing protocol for low-power lossy networks (RPL), and constrained application protocol (CoAP), certificate-based Internet security protocols have not been fully addressed yet.

We see the main cause of this being the size of the X.509-based Internet certificates. Typically being 1 to 2 kB, the large size of these certificates can cause IEEE 802.15.4-based IoT nodes to fragment the certificate into many smaller packet-size chunks, which causes many packet transmissions to occur in the network. This work presents *LightCert*, a lightweight scheme to compress the size of the security certificates using the similarity of contents in X.509 certificates. Specifically, *LightCert* identifies common fields in a certificate and suppresses the transmission of these contents within the IoT subnet scope. This allows *LightCert* nodes to minimize the packet transmission overhead for supporting certificate-based security mechanisms such as datagram transport layer security (DTLS), by as much as ~37%. The added overhead of exchanging certificates when using *LightCert* is kept low to as much as ~5 mJ of energy and ~0.48 seconds of latency.

1 | INTRODUCTION

The Internet of Things (IoT) and the devices consisting of the infrastructure are gaining interest from various industries as a future market that interconnects cyber intelligence with the physical world.¹ Given the number of applications that IoT systems can potentially cover, the number of devices that are deployed is increasing at an extremely fast rate and this rate is projected to grow even more rapidly over the next few years to come. As IoT devices tend to come closer to our everyday environments to capture personal information and analyze such information on the Internet for various (often third party) applications, the potential threat on personal information leakage increases as well. For such reasons, there are demands for implementing sufficient security mechanisms on IoT devices.

Specifically, as IoT devices can collect sensitive data, only authorized parties are allowed to access the data and manipulate it. As a result, it is essential to have a mechanism that can authenticate messages and the identity of the Internet- and IoT-connected devices. One of the most powerful and secure authentication technologies used on the Internet today is the public key infrastructure (PKI²). PKI is designed to operate based on an encrypted certificate by accessing a

public key via a private key possessed by a node considered providing high levels of security in terms of authentication and nonrepudiation.

However, the conventional PKI scheme is not efficient enough for the IoT environment given that many IoT-class devices have limitations in computing, network, and battery resources. Given the maximum packet size of the widely used IEEE 802.15.4 standard³ of 127 bytes, the PKI certificates are considerably large (eg, typically 1 kB to 2 kB). As the size of storage and memory is limited in many IoT devices, reducing the size of the certificates is preferred to save the space in general. If we apply the current PKI scheme to IoT devices, the running applications may be limited to populate data as well as to keep required certificates on memory. Such limitations can potentially lead to frequent certificate exchanges (due to the previous ones being deleted due to the lack of storage space), which results in additional increased network traffic and energy consumption overhead.

Sending a large number of packets for exchanging a certificate on a resource-constraint IoT device is challenging because of the amount of traffic and memory usage required to establish a secure session. IoT networks typically exploit a radio duty cycling link layer protocol⁴⁻⁶ to maintain a lifetime of up to several years on battery. Radio duty cycling can reduce the energy consumption of a node by minimizing the amount of time the radio transceiver is in the “on” state in accordance with the sleep and wake-up schedules. Such radio duty cycling will have a direct impact on the cost of certificate exchanges by increasing the latency and lowering the network throughput.

In this study, we propose reducing the size of the certificate to cope with the lack of resources. Smaller certificate sizes allow a resource-constraint IoT node to maintain more certificates in its memory, allowing it to communicate freely with many devices without the need to update or exchange the certificates frequently. In addition, the decreased packet count from the certificate size reduction can reduce the radio on time thus reduces the energy consumption of battery-operated IoT nodes. Furthermore, shorter packets lead to less congestion possibilities on the wireless medium, which can naturally save the resources needed for packet scheduling and retransmissions.

We introduce *LightCert*, a novel certificate compression and exchange mechanisms which are suitable for IoT devices and their deployments. Our proposed scheme first starts with compressing a typical Internet node certificate's formatting using the concise binary object representation (CBOR).⁷ Instead of redesigning the entire certificate specifically for IoT nodes, we leverage the certificates used on the Internet today to allow full Internet-compatibility in the IoT network. Next, borrowing from the philosophy of the 6LoWPAN^{8,9} header compression mechanism design, we exploit the capability to reconstruct a packet at the border router (eg, IoT gateway). Specifically, we compress the size of the certificate by eliminating common (shared) portions of the certificate within the subnet. While the certificate travels through the IoT network, we make sure that implicitly understandable fields are suppressed, but as packets *leaves the IoT network*, the gateway attaches the suppressed information to packets to construct a full-size certificate. Given that our scheme focuses on maintaining the *contents* of the certificate while only compressing the size, we maintain compatibility with the certificate-based packet exchange mechanisms used on the Internet today.

We evaluate the effectiveness of *LightCert* in terms of network overhead (generated from supporting certificate-based security) and energy consumption using a combination of ContikiOS implementation-based experiments and NS-3-based simulations. Our results suggest that compared to exchanging a full-length certificate, *LightCert* is able to reduce the network packet exchange overhead by as much as 37% and the energy consumption by 30%. Furthermore, the increased energy usage overhead, when compared to the case where no security mechanisms are used, is only 5 mJ while testing for a typical IoT application scenario.

Specifically, this work makes the following contributions.

- We present a set of requirements that developers must consider in designing security mechanisms for resource-constraint IoT platforms.
- We design and implement *LightCert*, a light-weight algorithm for compressing and exchanging certificates for enabling PKI in IoT systems.
- Using a Contiki implementation of *LightCert*, we show that the proposed scheme can effectively reduce the packet exchange overhead that security certificates introduce to minimize energy usage on resource-constraint IoT systems.

The remainder of this paper is structured as follows. Section 2 covers related work on various emerging IoT security technologies, and Section 3 introduces background information on certificate-based security requirements for the Internet and IoT. In Section 4, a detailed description of *LightCert* will be presented, and we will present evaluation results in Section 5. Finally, we conclude the work with an overview of the remaining challenges and future directions for IoT system security in Section 6.

2 | RELATED WORK

Over the last few years, there have been many proposals for building a customized system for secure IoT networks. Most of these approaches are partially dependent on remote and centralized cloud servers for IoT node authentication and authorization.^{10–13} OpenIoT is a platform designed to enable integration between different sets of IoT applications.¹⁴ It is based primarily on cloud servers and leverages central access control to adopt a flexible authentication and authorization scheme. Authentication of Things (AoT) provides a variety of authentication schemes and uses a cloud server to control the trust relationship between the manufacturer and the IoT device.¹⁵ IoT-OAS uses the OAuth server with the CoAP protocol for access control to third-party applications,¹⁶ and OSCAR uses an authentication server to provide an integrated security architecture for the cloud and IoT systems.¹⁷ Another paper has established a new IoT-Cloud architecture based on edge computing to overcome internal attacks and repeat/similar service requirements. It focuses on ensuring IoT security and addressing service efficiency issues.¹⁸

The SEA¹⁹ adopts gateway to supplement the limited resource environment. This solution uses a distributed smart electronic health gateway to present an authentication and authorization architecture for secure and efficient IoT-based healthcare. It entrusts the burden of the healthcare sensor node to the outside, allowing the sensor to communicate securely and efficiently across independent network domains. By providing the connection context set in the health sensor node, no longer need to authenticate and authenticate the remote health center or care giver. The SEA uses the fact that smart electronic health gateways and remote end users have sufficient resources to efficiently perform various weight protocols and certificate validation.

Kothmayr et al propose an authentication system for IoT using datagram transport layer security (DTLS), a datagram variant of transport layer security (TLS),²⁰ and Hummen et al propose a security framework for IoT devices based on DTLS.²¹ As with our proposed approach, these frameworks use a delegated server, which is an authorization entity, to reduce the amount of public-key cryptography computations at the resource-limited end devices.

In addition, there are ways to keep the conventional authentication format but reduce the size of the certificate itself to reduce the traffic for a wireless sensor network channel.²² This technique proposes a new formatting certificate using CBOR rather than ASN.1, which is conventionally used in X.509 certificates.

3 | SECURITY ON THE INTERNET AND REQUIREMENTS FOR IOT DEVICES

3.1 | Certificate-based internet security

IoT devices installed in the home, hospital, and factory enable many application services. Naturally, given that these nodes can carry sensitive information that directly relates to privacy and safety, the networks need some form of security measures to protect their data. These security mechanisms are necessary not only for the users safety but also required for protecting others who access data from the device if the device is accessible from third parties and shares meaningful data. To do so, the main focus is in identifying potential risks and finding ways to ensure that threats related to data leakage are suppressed. Nevertheless, it is not easy for the system to stay on top of everything under dynamic environments; thus, typical security-/privacy-preserving mechanisms rely on an externally trusted Internet entity.

Currently, the most commonly used security method on the Internet is the Public Key Infrastructure (PKI), which exploits *certificates* to maintain the authenticity of connections. Using PKI makes it possible for public network users, such as users on the Internet, to exchange data steadily and confidently by using a certificate granted by a trusted authority. PKI provides a digital certificate that identifies an individual or organization, and a directory service that stores the certificate and can be retrieved when needed.²

A certificate is a digital system or code that is responsible for data encryption, authentication, and verification. A certificate of authenticity can be issued by a certification authority (CA) that can be trusted. CAs must adhere to very strict rules and policies when issuing certificates and are managed under government regulations. If a device has a certificate issued by a trusted CA, the device and its partners can be trusted; thus, they can exchange data in a secure manner.

PKI exploits public key cryptography, the most common method used for authenticating message senders over the Internet or for encrypting messages. Traditional cryptography is often involved in creating and sharing “secret keys” that are used to encrypt and read messages. These secret (or private) key systems possess a fatal weakness in the fact that they can be easily intercepted from the network. For this reason, PKI is preferred in the current Internet structure.

TABLE 1 ROM and RAM sizes list of various MCUs used for IoT systems

MCU	Flash	RAM
MSP430F1611 (Texas Instruments ²³)	48 kB	10 kB
Cortex M0+ (Atmel ²⁴)	256 kB	32 kB
Cortex M3 (Atmel ²⁵)	256 kB	52 kB

3.2 | Certificate requirements for IoT devices

While maintaining the aforementioned security levels is attractive, unfortunately, the conventional PKI scheme is not suitable for verifying data generated by lightweight terminals (eg, resource-limited IoT devices). Specifically, the typical size of certificates exchanged in the existing Internet is 800 to 2000 Bytes. This is considerably large considering that the maximum frame size of the widely used IEEE 802.15.4³ standard is only 127 bytes. This is a significant burden to the IoT network, in which nodes keep their radio off, and periodically wakes up to exchange data as a way to improve the nodes' battery efficiency. Although revisions of the IEEE 802.15.4 protocol such as the 802.15.4e standard proposed expansions that relaxes the per-frame size limitations and allow the transmission of more data in a single packet, we focus on the original IEEE 802.15.4 protocol given that they are still more widely used compared to the revised protocols.

The certificate size not only affects the network performance but also impacts the limited storage capacity of IoT devices. Table 1 shows the RAM and flash sizes of several popular microcontrollers (MCUs). These numbers indicate that for the widely used TI MSP430 and ARM Cortex-series MCUs, the amount of RAM and flash memory available for certificate storage is very limited. Furthermore, the limited space is stressed by the actual data gathered from the devices' sensors, and the essential code stack for IoT system operation. Together with the network and application implementations, the size of the certificates, and indeed the size of the program code to process the certificates, can utilize only a small amount of space.

Based on these observations, we present a set of requirements that must be considered when implementing certificate-based security to IoT networks.

- **Memory size/usage requirements:** The limited memory and storage capacities on the low-power, resource-limited nodes used in IoT systems force the system developers to focus only on a limited scale of security operations. Furthermore, with certificates used in the IoT security mechanisms, the number of certificates a single node can store is naturally limited by a node's Flash size. Therefore, not only should we support a lightweight *scheme* for IoT security, but we should also minimize the size of certificates we target to utilize. In doing so, we should initially try to maintain full compatibility with the Internet architecture and also present schemes that are more optimized for use in the "Intra-net" of things.
- **Energy requirements:** Low-power is key to many IoT systems. Many devices operate on batteries with a tight lifetime target and radio low-duty cycles.⁴ Applying security schemes to IoT devices should not (at the very best) increase the power usage. This power usage includes the computational time to secure the packet (and links), and also include the power caused by transmitting longer packets for security purposes.
- **Network traffic requirements:** Related to the energy requirements discussed above, the increased packet size can force the radio to stay up longer and increase the energy usage of battery-operated IoT nodes. Furthermore, longer packets lead to more congestion possibilities on the wireless medium,²⁶ which can naturally introduce additional costs due to packet scheduling. Supporting certificates in IoT platforms should take in consideration that including such context in the packet directly can cause the packet sizes (and potentially the quantity due to message fragmentation) to increase.

With these base requirements, we present our scheme to effectively support certificate-based security on IoT devices in the following sections.

3.3 | TLS and DTLS operations

TLS²⁷ and DTLS²⁸ are the most popular protocols used for enabling PKI-based security structures. Therefore, *LightCert* is designed based on the use of TLS/DTLS. In the TLS/DTLS handshake process, if the client and the server agree on the certificate key exchange process, the server sends a certificate message for authentication. The client may want to re-establish the connection by sending a `client hello` using the existing session ID. If the session ID sent by the client matches the session cache of the server, we can know that the purpose of this exchange is only to re-establish a previous connection. Otherwise, the pairs will establish a new session ID and execute a full handshake process.

Traditional networks can have long-living TLS/DTLS connections, but in the IoT domain, there is a high chance that these connections will only last for short periods. Specifically, IoT devices typically have a tight limit on the amount of memory space they can hold. In the TLS/DTLS process, the client and server nodes need memory space to store the certificates and associated session values to re-establish prior connections. However, with space limitations, all information cannot be stored on the IoT node; thus, the certificate and its associated data may be deleted, for the sake of storing other certificate data and can result in a continuous exchange of certificates with the same party.

We can also think of the IoT scenario where various users connect for a short time only to get simple information from the nodes. For example, we might assume that users are getting information about a parking place in a public car park. This is a common scenario for publicly-deployed IoT systems. Such a situation does not require the node to store the user's certificate consistently, but since new connections are made frequently, the certificate exchange process often occurs as well.

3.4 | Threat model

The development of the IoT network has a tremendous impact on the spread of the Internet in various fields. This provides great convenience to people, but at the same time provides an opportunity for hackers to attack. A fragmentary example is when a hacker masquerades as a node and breaks into the IoT subnet. If all nodes in the network are free to exchange information with third-party devices, users can be in a very dangerous situation.

LightCert targets to make improvements to the exchange process of certificates for authentication between devices sending and receiving real data. For this, we assume that the gateway device can be trusted. Since we take a scenario where the IoT is first installed by the users themselves, we do not see this as a strong assumption. Even if the gateway was somehow considered to be malicious, since *LightCert* assures end-to-end reliability, the contents can still be protected. Therefore, while gateways are reliable, data exchanging entities are not fully trusted. In addition, all discussions on *LightCert* presume a state where the nodes are already issued a certificate from an Internet entity that is sufficiently trusted (eg, authorized CAs).

4 | LIGHTCERT

Based on the requirements for a lightweight certificate for low-power IoT, as we mentioned in the previous section, we now discuss in detail on the design of *LightCert*, our proposal for supporting Internet-level certificates on resource-limited IoT platforms. Specifically, the contents of the certificate exchanged to establish a secure connection become a burden to the IoT devices and network packets, given that IEEE 802.15.4-based low-power networks can only handle packet sizes of up to 127 octets in the payload.³ Having to deliver large certificates leads to the need for packet fragmentation, increasing the number of packets to be transmitted even for exchanging a small amount of “valid” information. This naturally leads to resource drain, both in terms of bandwidth and energy.

LightCert offers a lightweight mechanism for handling security certificates on low-power resource-limited IoT devices (typically 6LoWPAN/IPv6-based) by exploiting the two following facts. First, we transform the X.509 certificates²⁹ in ASN.1 format to a *reduced format* based on the concise binary object representation (CBOR) format.⁷ The CBOR format offers a lightweight structure, which allows the original ASN.1 byte sequence to be represented using a smaller number of bytes. We have attempted to make the certificate structurally lightweight by using the relatively free CBOR format rather than the traditionally used ASN.1.

Second, we utilize the capability of the gateway (eg, 6LoWPAN border router) and hashing to minimize the payload exchanged over the wireless network channel. Specifically, in *LightCert*, we adopt the fact that neighboring IoT nodes that are typically bounded to the same subnet (eg, devices sharing the same 6LoWPAN border router) share many similarities. In fact, this similarity in network configurations is what makes the 6LoWPAN IPv6 header compression process possible. In 6LoWPAN, common (and preknown) portions of the IPv6 header or items that can be extracted from other parts of the message such as `version`, `payload length`, and `traffic class` are eliminated from the packet that is transmitted from the link layer. The 6LoWPAN border router takes the role of “reconstructing” the header if the packet needs to leave the 6LoWPAN subnet. As for certificates, similar to the 6LoWPAN approach, there are many common contents that can be shared within certificates for nodes in the same subnet. We can easily assume that nodes on the same subnet are typically issued certificates from the same or a chain of CAs, and they also share similarities in the extension section of the certificate due to their similar purposes. Therefore, instead of having each resource-limited IoT node issuing and

forwarding packets with the full-sized security certificate, we implicitly omit common information and send only the content that is “unique” to the specific node within the IoT-nodes’ link layer transmissions.

To allow this, the gateway (eg, 6LoWPAN border router) oversees the process of managing the full certificate. Gateways provide a mechanism for all nodes in its subnet to easily discover common parts within the certificate, which are then used to facilitate easy certificate-size reduction within the IoT network. If all nodes generate the key values with a uniform hash function shared in the same gateway’s subnet scope, these keys can be used to compare mutual certificates. The common parts obtained through the comparison process do not need to be explicitly sent; thus, this can reduce the payload that an IoT node must send for maintaining a secure channel. Note that we focus mostly on the efficient transmission (and reception) at resource-limited IoT devices and their link layer connections. The goal of *LightCert* is to assure that the IoT nodes conserve their resources as much as possible while maintaining standards-compliant connectivity when reaching the conventional Internet architecture. The following subsections will detail our proposed mechanism.

4.1 | Reduced format

The focus of the reduced format, which is a modified version of the ASN.1-based certificate, is to achieve efficient reassembly and lightweight exchange procedures. Even if the format is changed for lightweight operations, the certificate must be able to be restored to its original form to guarantee the integrity. The majority of the X.509 certificate format follows strict ordering, but the `extensions` field can be randomly arranged. Therefore, even when the certificate is reduced in size, the reduced certificate must still maintain the orderings of the original (nonreduced) certificate, including the original structure of the extensions field. To take a simpler formatting approach than ASN.1, *LightCert* reduces the certificates’ size by using the CBOR format.⁷

Each attribute in the reduced certificate consists of `type`, `size`, and `contents`. `Type` is used as an indicator of the content type itself (eg, issuer, subject, validity period) from the original certificate. The object identifier (OID) that is typically included in X.509 certificates is also represented using the `type` attribute. `Size` describes the length of the following content, and it is an indicator of the exact bounds of each attribute. If the value of `size` is 0, it means that the corresponding attribute values of both certificates are the same. In this case, only the order of the certificate is defined, and the contents are not transmitted. Finally, `content` holds the actual data of the attribute.

4.2 | Certificate registration

We present in Figure 1, the process of registering a certificate to the gateway. All nodes first connect to the gateway and then request their certificate to be hashed at the gateway (Figure 1A). The PKI certificates currently being used have a fixed format tailored to the certificate’s version. To reduce the certificate size, the gateway operates a hash function, which makes use of the subdivided certificate format as input parameters, except for the essential portions such as the public key and signature (Figure 1B). Using the output of the hash, the gateway maps the key to the input structure and stores this information in the “Field” table (Figure 1C). The “Field” table guarantees that the key values shared among the nodes in the subnet come from the same certificate fields. This process also transforms the certificate into a *reduced format*, maps to the IP address of the node, and stores this data in the “Complete” table (Figure 1D). The “Complete” table is managed

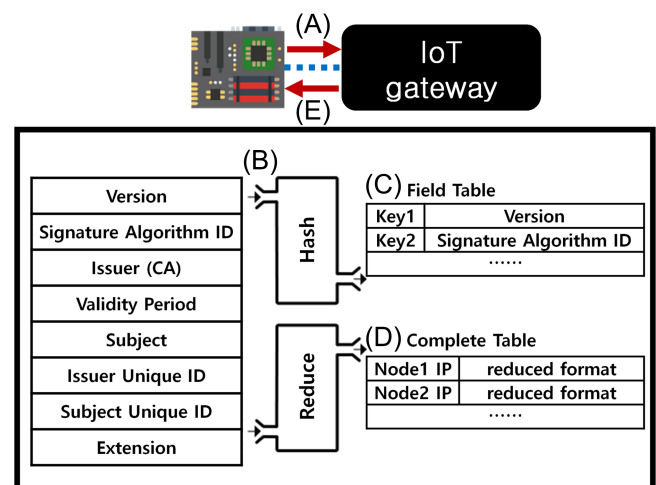


FIGURE 1 The process of registering a certificate, which takes place when a node is connected to a gateway. IoT, Internet of Things

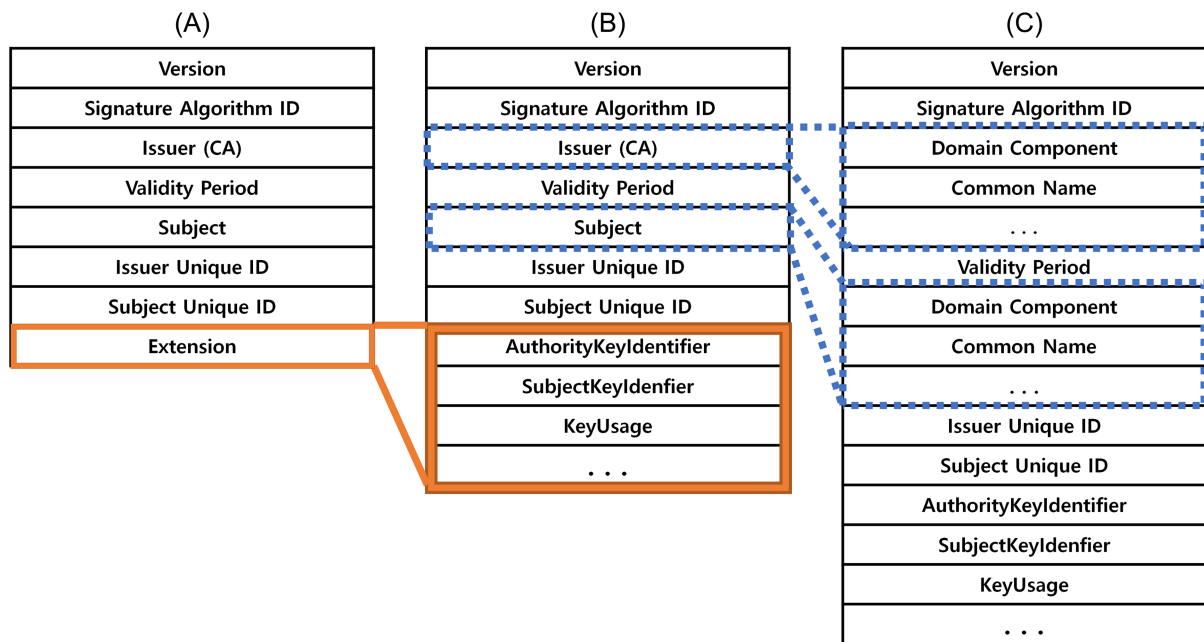


FIGURE 2 The degree of certificate fragmentation to be used as the prehashing argument value for different compression levels. A, Level 0; B, Level 1; C, Level 2

at the gateway to proxy the transfer of an IoT node's certificate. Finally, the node obtains a reduced format of the original certificate, which is created using its own unique full certificate (Figure 1E).

LightCert defines three levels of certificate compression when dividing the arguments for the hash function. Specifically, we take a closer look at X.509 certificates as depicted in Figure 2. When the gateway hashes the certificate in “Level 0,” it splits the certificate based on the basic fields (Figure 2A) of the original certificate. If the content of a specific field matches for two certificates, the common field can be compressed. In “Level 1,” the extension field is further divided into multiple subfields (Figure 2B). Therefore, even if the extension field does not fully match, since “Level 1” performs hashing based on the subfields separately, subfields that are common can be compressed. This fine-grain division allows the two certificates to compress the certificate more efficiently compared to “Level 0.” Finally, “Level 2” will divide the certificates fields at a finer scale (Figure 2C). Overall, as the level of compression increases, the certificate is further subdivided as it passes through the hash function. Operationally, this will allow *LightCert* to identify more common parts in a certificate to further compress the certificate. On the other hand, the use of more complex hash operations can cause transmission overhead in the hashing key list exchange phase when the client node sends its information to the server for certificate compression.

4.3 | Handshaking

A handshaking process is required to form a secure virtual private network (VPN). During the hand shaking process, typical TLS/DTLS systems exchange information on what characteristics the VPN connection will hold (eg, cipher suites). First, if the server node (the node receiving the communication request) also supports *LightCert*, the two nodes will trade their hashing key values to identify what parts of the certificate are common and what parts are different. This hashing key consists of the list of hash function keys provided from the *LightCert* gateway. Following this step, the reduced certificate of the server will be sent back to the client. Timingwise, as Figure 3B shows, the hash key exchange and server certificate exchange will take place after then SeverHello message, when the connection is made between the node pair. As a result, all nodes that support *LightCert* can easily identify what different contents exist in their key/certificates, and thus, this content can remain as the parts to be transmitted, as a way to reduce the overhead of exchanging full-size certificates.

In the second case, in which the server node does not support *LightCert*, the node will reject the use of a *LightCert*-based VPN connection. Naturally, the certificate exchange procedure will fall back to the traditional TLS/DTLS method as depicted in Figure 3A. Even so, we note that the following section explains how certificate transmissions within the IoT subnet can be done more efficiently.

FIGURE 3 In the Internet-of-Things (IoT) subnet, each node transmits a hashing key list to compare the commonality prior to certificate exchange. ClientHello notifies the use of *LightCert* and exchanges key values and the reduced certificate after the ServerHello. A, Original; B, *LightCert*

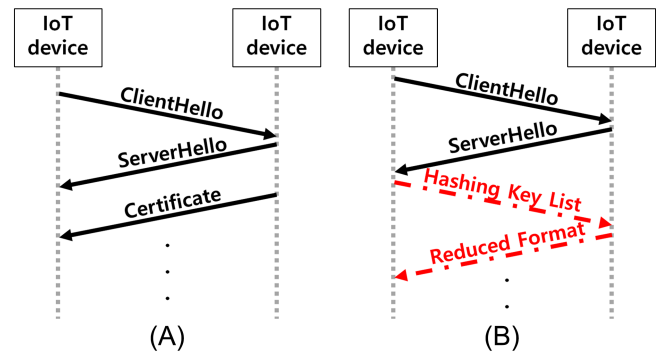
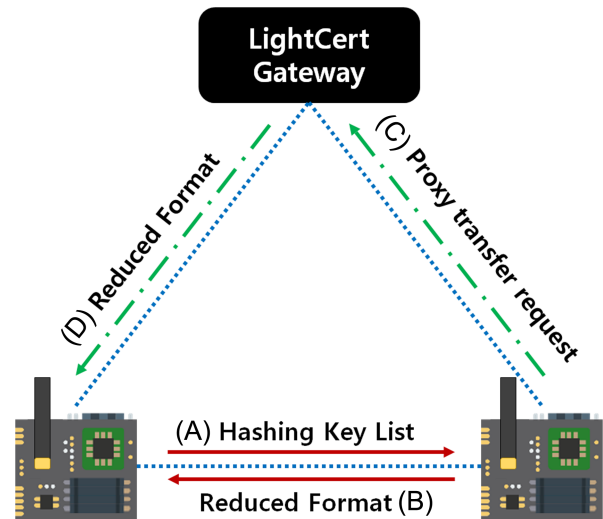


FIGURE 4 Certificate transmissions occurring within the Internet-of-Things subnet



4.4 | Certificate exchange

For the certificate exchange process, we consider three possible scenarios. First, we assume the exchange of information between IoT devices in the same IoT subnet. The second regards data transfer (outbound) from an IoT node to the external network, which can occur in situations where information is requested from a server or external individual. The last is the receipt of the certificate from the outside of the IoT subnet to the inside (inbound).

4.4.1 | IoT subnet scenario

The process of exchanging certificates in an IoT subnet connected to the same gateway (first scenario) is shown in Figure 4. We consider an IoT subnetwork which utilizes the IETF routing protocol for low-power lossy networks (RPL³⁰). RPL has two modes of operation: *nonstoring mode* and *storing mode*. In the *nonstoring mode*, all traffic must go through the gateway to transmit data between nodes, whereas *storing mode* can allow direct node-to-node communication without gateway support. In *storing mode*, client nodes can communicate without the help of a gateway, so the hashing key list must be sent (from the source node) to the server (ie, destination IoT node) to notify it of the commonalities in the certificates (Figure 4A). This server node identifies the differences between the two nodes' certificates using the key list, then creates a reduced certificate format of its own (tailored for the source node), and responds with a *ServerHello* message to the client (eg, source node) (Figure 4B). It is meaningful to point out that using *storing mode* can induce memory overhead. Nevertheless, since *LightCert* introduces three levels of certificate compression, suitable for different system configurations, systems with sufficient memory can select to use the lowest level of compression with less computational overhead, while devices with strict memory limitations can exploit other compression levels with the sacrifice in computational complexity. On the other hand, when using the *nonstoring mode*, since the *LightCert* gateway stores the certificates of all nodes in its subnet along with their key information, the server node requests a *proxy transfer* to the gateway when the certificate needs to be transmitted (Figure 4C). When the *proxy transfer* request is received, the gateway will transmit the reduced-size certificate of the server node on its behalf (Figure 4D). The client that obtains the reduced

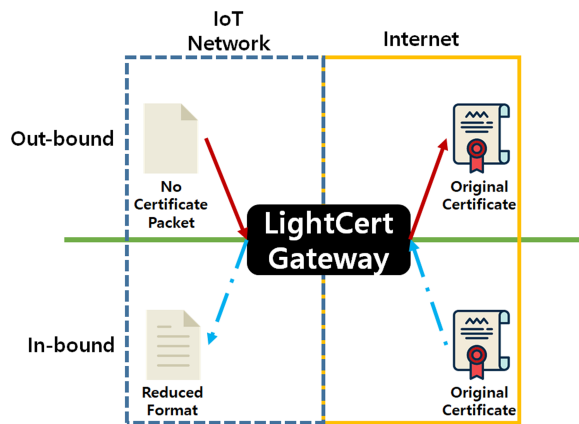


FIGURE 5 The *LightCert* gateway operations in the import and export operations of certificates between the Internet-of-Things (IoT) network and the Internet. Through this process, IoT networks can reduce the data transmission burden

certificate reassembles the server's certificate information by combining the contents of its own certificate. We note that the complexity of the certificate reduction is minimal given that there are no complex operations occurring in the process.

4.4.2 | Outbound certificates from the IoT subnet

In the second scenario, we consider the case when an IoT node sends its certificate to nodes in external networks. Here, the wireless channel of the IoT network can experience a significant amount of overhead when delivering (large-sized) full security certificates as the packets that carry this information pass through the IoT network. To address this inefficiency, *LightCert* configures the IoT gateway to play an important role by maintaining all nodes' certificates within its subnet. In other words, nodes that receive external data requests and need to send a copy of its certificate to an external destination create an empty certificate packet to send to the gateway. When the gateway receives this message, the gateway assembles the IoT nodes' full certificate to the packet and passes this information to the final destination. Note that the packet can be signed at the IoT node while "assuming" that the packet's full contents, including the certificate, are in place, and then, the certificate information is removed from the packet to reduce overhead "within" the IoT network. This process allows *LightCert*-based IoT networks to minimize the overhead caused by certificate transmissions (Figure 5 outbound).

4.4.3 | Inbound certificates to the IoT subnet

The last scenario is a case where an external node's certificate enters the IoT network. The *LightCert* gateway obtains an external certificate from the ServerHello packet received from the external network. Then, this certificate obtains the key value from the hashing function applied in the network, compares the result with the reduced format of the destination node IP, and sends the reassembled certificate with only the necessary parts to the destination node (Figure 5 inbound). This is clearly different from the first scenario. In the first scenario, the similarity between nodes can be sufficiently predicted through the characteristics of the IoT subnet itself. However, in the case of incoming external certificates, it is difficult to measure the degree of similarity. IoT subnets with similar use cases may share a high degree of similarity, but a node with the completely unrelated function may be difficult to find a commonality. Using the three aforementioned scenarios, we test the effectiveness of *LightCert* in the following section.

5 | EVALUATION

We evaluate the performance of *LightCert* using a combination of NS-3-based simulations and ContikiOS-based real-world experiments. Specifically, we use the simulation results to observe the operations of *LightCert* when the system scales to many resource-limited IoT nodes in various scenarios and uses the real-world experiment to validate the protocol operation and energy efficiency of *LightCert*. We understand that network simulations can only partially reflect reality but still find it useful to make predictions on the system performance in settings that are difficult to implement in real world.

Specifically, the NS-3 simulation environment is divided into two networks, in which the first is the low-power IoT network using IEEE 802.15.4 networks and the second is a wired Ethernet-connected network. The IoT network uses the 6LoWPAN standard as provided by NS-3, and the add-on implementations are done on our side for successfully supporting *LightCert*. The per-node processing time for the packets at each node was empirically gathered using our ContikiOS implementation of *LightCert* on Zoul Firefly sensor motes. This platform is based on an ARM Cortex-M3 MCU

with 512-kB flash, 32-kB RAM, and a CC2538 radio chip. We use this device given that it sufficiently satisfies the memory requirements for the Contiki IP stack. All embedded implementations are done using the UDP, 6LoWPAN, CSMA, and IEEE 802.15.4 implementations of ContikiOS. In addition, we adopted the ContikiMac as radio duty cycle (RDC) layer,⁶ which is the MAC protocol for energy conservation of radio transceiver. Since the radio component is the primary source of power consumption in IoT devices, we monitor the performance of *LightCert* under scenarios with RDC. Note that we use the log distance path-loss model with the path-loss exponent γ as 2, reference distance d_0 as 1 meter, and reference loss PL_0 as 50 dBm. Link layer retransmissions are used to support the minimal packet loss occurring on our network configuration. Wired connections within the simulation environment are configured to have no path loss.

5.1 | Certificate sets

Given that the CA can issue different types of certificates, we use four different certificate sets for evaluating the performance of our scheme: the **subnet set (SS)** represents a set of certificates that maximizes certificate similarity by assuming that the certificates are allocated to nodes within the scope of a single-purposed IoT subnet like a factory. Note that the certificates that are tailored for IoT nodes do not yet exist ubiquitously; thus, we make a logical assumption on the structure and take observations from customized IoT node certificates used in systems such as Azure IoT³¹ and AWS IoT³² to design the certificates in SS. Specifically, the certificates in SS are similar in all aspects except for some subject DN, extension, and the key algorithm (eg, Subject commonName, Subject emailAddress, extension SubjectKeyIdentifier). The **diverse certificate set (DCS)** considers diversely purposed IoT devices that may have some common characteristics but can have different subject and extension fields and even different issuers. The DCS assumes that there is *another* IoT network with a similar purpose that supports *LightCert*, or a smart home where various un-unified devices can be installed on the same network. Discrimination in issuers takes into examination the situation where various devices are deployed for different purposes. The certificates in DCS are also customized based on common IoT system requirements and the IoT certificates used in Azure and AWS. As Figure 6 shows, the certificates in SS (depicted in solid red lines in Figure 6) are limited in their contents, given that these certificates are designed for single purpose systems. On the other hand, given that DCS certificates cover a broader set of applications, their contents are richer compared to SS (depicted in blue dotted lines in Figure 6). When creating the SS and DCS certificates, we use SHA256 signatures and use ECC and RSA public key algorithms interchangeably for different certificates in the SS and DCS certificate sets. Finally, certificates in the **conventional set (CS)** represent a set where we consider a certificate exchange with a conventional network to which *LightCert* is *not applied*. The CS evaluations show how much an existing PKI certificate can be reduced using common key values generated at the SS certificates. Existing PKI certificates used in CS are either imported from popular sites or created with reference to them.

Table 2 presents the size and number of certificates that consist of each certificate set used in our experiments. Although the certificates included in the SS have a high degree of similarity, the reason why the difference between the min/max sizes is large is due to the differences in the public key and signature fields among various SS certificates. Furthermore, the average size of SS and DCS is smaller than CS certificates (common Internet node certificates), given that the SS and DCS certificates were created by minimizing the certificate's contents by assuming an IoT environment operation.

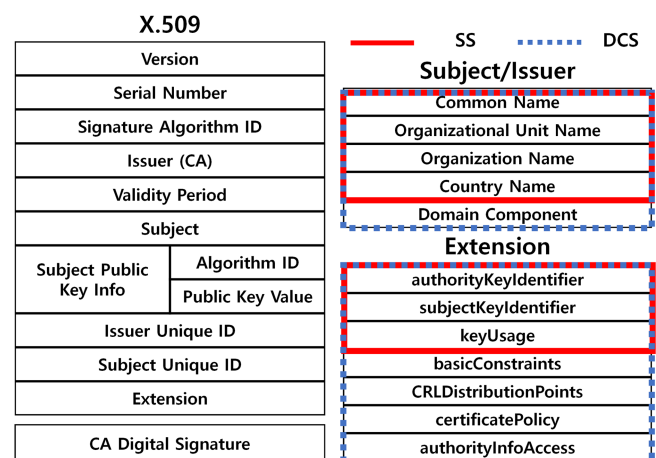


FIGURE 6 The components in the certificates that make up the subnet set (SS) and subnet set (DCS). CA, certification authority

Set	Size (bytes)	Number
SS	742-1170	15
DCS	796-1304	30
CS	1194-2369	10

TABLE 2 The number and min/max size of the certificates that consist each certificate set used in our study

Abbreviations: CS, conventional set; DCS, diverse certificate set; SS, subnet set.

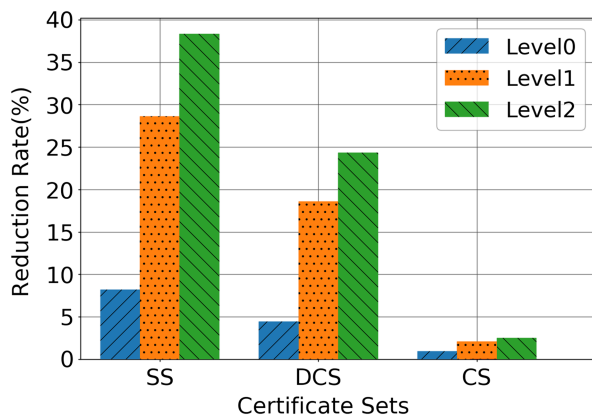


FIGURE 7 Certificate size reductions achieved by *LightCert*. For subnet set (SS) certificates, which have the most commonality, certificates are reduced by up to 37%. CS, conventional set; DCS, diverse certificate set

5.2 | Certificate reduction rate

We present the result of the certificate compression process for the certificate sets discussed above in Figure 7. This experiment fills the hash table with SS and DCS certificates and then calculates the average of the reduction rates. CS certificates were excluded from completing the hash table to study the input from the sender that was *not* associated with the IoT network. By observing the results, notice that the maximum reduction rate of the certificate is ~37% for the SS. Specifically, for CS and DCS, most of the certificate contents are compressed except for the public key and the signature portions of the original certificate (given that these two fields are different for many of the certificates). In contrast, CS is only minimally compressed given the limitations in the commonness among different certificates. Certificates in the SS class, as expected, show the most reduction in certificate size, whereas certificates with less similarity (ie, DCS and CS) show smaller benefits of compressing (or omitting) common parts in the certificate contents. The certificate contents that are omitted using Level 0 compression include fields such as the Issuer or AlgorithmID, and from using Level 1, we can reduce parts of the extension fields. Finally, from Level 2, *LightCert* reduces the DistinguishedNames field from the original certificate.

5.3 | Energy consumption rate and latency

We conducted experimentation with Zoul Firefly nodes and the NS-3 simulator to observe how efficiently *LightCert* reduces the energy consumption and latency when supporting and exchanging certificates. We vary the compression method and the network size (eg, hop count from the source to the destination) in our experiments. The Zoul Firefly node was used to measure the actual computational latency and energy usage rate of different operations via monsoon power monitor³³ and we combine real node-level measurements with NS-3 simulation traces to compute the energy usage and certificate exchange latency in different network scenarios.

To transmit a certificate, it is necessary to split the certificate into units of IEEE 802.15.4 fragments. The space available for transporting actual payload data is 70 to 80 bytes in a 127-byte packet, excluding additional information such as the PHY, MAC, and IP headers. Therefore, transmitting a packet with certificate information typically requires, even when compressed, a series of packet transmissions rather than a single packet exchange. When using a low-power MAC (eg, RDC), this can result in long latency given that typically a single packet is transmitted in a single cycle. Luckily, the ContikiMAC used for our measurements supports *link-layer bursts*³⁴ for energy-efficient successive multiframe transmission. Specifically, instead of having a sender wait for the receiver's wake-up for each frame transmission, link layer burst inserts a pending bit in the frame header to notify the presence of another pending frame transmission. As a result, the receiver will stay awake to receive the message directly after the first packet exchange; thus, reducing the latency of transmitting multiple packets. We take these practical factors into account when setting up the NS-3 simulation network.

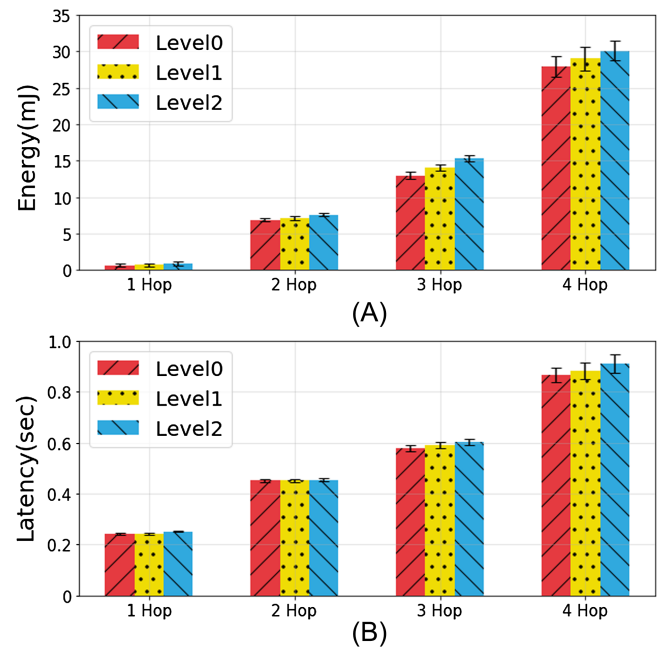


FIGURE 8 Energy/latency overhead in exchanging hashing key lists between certificates in the Internet-of-Things subnet. A, Energy; B, Latency

In our evaluations, where we test for the exchange of SS, DCS, and CS certificates, we note that we focus on the “internal” IoT network performance. That is, while DCS and CS certificates interact with the IoT subnet *and* the external Internet architecture, our evaluations focus on the latency and energy usage within the IoT subnet only. The distance that the certificate travels within the IoT subnet is varied from 1 to 4 hops on a network topology with 20 randomly positioned nodes. To create a level of contention in the network, we set all nodes other than the source and destination nodes to periodically issue a broadcast packet every 6 to 9 seconds. Finally, we do not introduce link level loss due to signal attenuation to solely focus on analyzing the system performance of a well-provisioned network. The result is shown by the following graphs that presents mean and standard deviation as bar and error bar.

Figure 8 plots the energy consumption rate and latency overheads of exchanging the hashing key lists within the IoT subnet. While Level 0 shows the best performance and Level 2 indicates slightly higher overhead levels, overall, the energy and latency overheads that occur during the transmission of the hashing key list tend to be similar for all compression levels. The two overheads increase in proportion to the hop count, while the energy rises up to 25 to 30 mJ and latency 0.8 to 0.9 seconds. The nonlinearity in the plots of Figure 8 is an impact of retransmissions in the lossy IoT environment. While latency and energy overhead on a networking perspective for the different compression levels are minimal, they can still be significant for tight-energy-budget systems. Furthermore, using Level 0 compression will still be beneficial in easing the memory overhead when storing the hashing key lists at the gateway and individual IoT nodes. Therefore, this compression level is a system-level design choice that IoT system designers can select.

Figure 9 shows the per-certificate transmission energy consumption within the IoT network. Here, we target to observe the networking impact of delivering a certificate packet(s). Since the original certificate size is similar for SS and DCS (Table 2), there is no significant difference in energy usage for exchanging these original certificates in the two cases (compare the “original” plots in Figure 9A,B). On the other hand, the size of CS certificates is larger than others; therefore, the energy consumption of exchanging the original certificate exchange is much higher. When the source and destination nodes are a single hop away, the performance does not differ noticeably compared to the original certificate. However, as the number of hops increase, SS and DCS tend to show a significant amount of improvement when compressing the certificate, while given that the CS certificate compression rate itself is low, CS does not show a dramatic improvement. Quantitatively, for SS Level 2 certificates, the energy savings were approximately 35 mJ for a single certificate exchange over 4 hops and DCS Level 2 certificates saved 30 mJ in the same 4 hop scenario.

Figure 10 plots the latency for the certificate exchange scenario over a varying number of hops from the source (client) and destination (server). Similar to the energy consumption plots in Figure 9, the certificate exchange latency for SS and DCS certificates increase with the hop count. The increasing trend for SS certificates are less significant compared to the DCS given the more effective certificate compression rate, thus transmitting a smaller number of packets and reducing the chances of channel collisions.

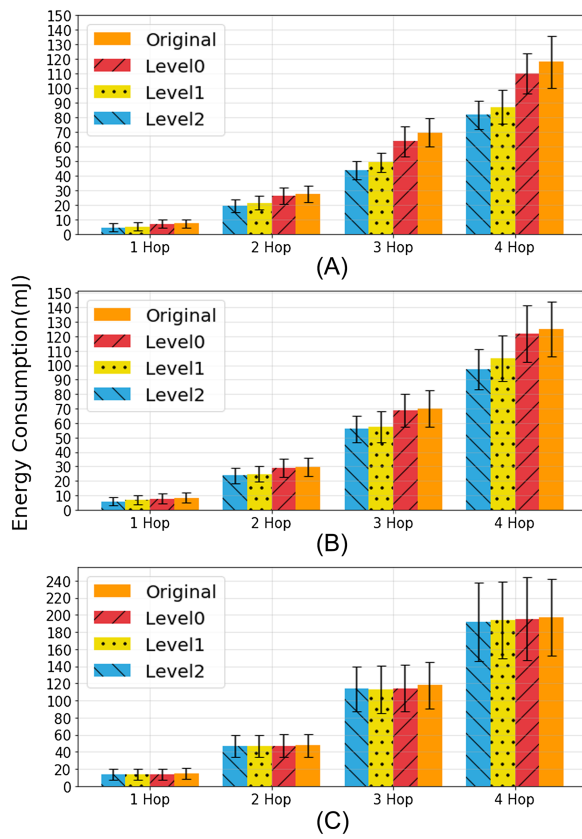


FIGURE 9 Energy consumption rate caused from transmissions for (A) subnet set (SS), (B) diverse certificate set (DCS), and (C) conventional set (CS)

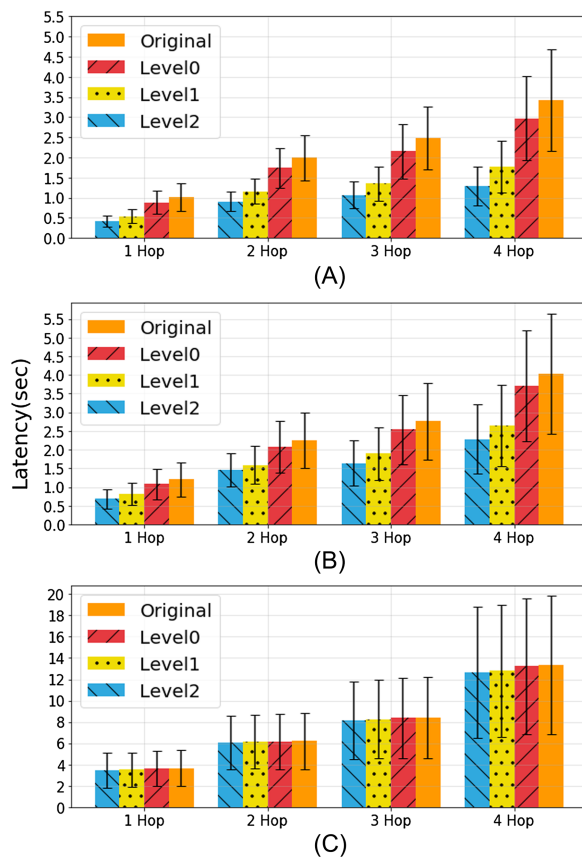


FIGURE 10 Delay for transmitting the *reduced format*. A, Subnet set (SS); B, Diverse certificate set (DCS); C, Conventional set (CS)

Overall, our experiments show that *LightCert* offers an effective way to compress Internet certificates for IoT nodes, which possess network bandwidth and energy limitations.

6 | CONCLUSIONS

Internet-of-Things devices possess unique challenges on a system design perspective by introducing battery and network limitations. As a result, despite having suitable Internet Protocol standards, such as 6LoWPAN or RPL, utilizing the security certificates used in the traditional Internet architecture directly remained as a difficult task. This work identifies such limitations and targets to design a simple and lightweight algorithm to compress the security certificates to be suitable for resource limited IoT nodes. Specifically, we present *LightCert*, which focuses on the similarities of the X.509 certificate contents. By suppressing the transmissions of certificate contents that are common across many certificates, the original certificates are reduced by up to ~37% of the original. This reduction naturally leads to reduction in energy usage and data transmission latency overhead. Our study shows that with a compressed certificate, IoT nodes can also join the Internet security architecture with minimal overhead. We foresee this work as a first step in developing various mechanisms for minimizing Internet security certificates for IoT nodes.

ACKNOWLEDGMENTS

This work was supported by the MOTIE and Korea Institute for Advancement of Technology through the International Cooperative R&D program and by the Basic Science Research Program through National Research Foundation of Korea funded by the MSIT (2018R1C1B6003869).

ORCID

JeongGil Ko  <https://orcid.org/0000-0003-0799-4039>

REFERENCES

1. Perera C, Liu CH, Jayawardena S, Chen M. A survey on internet of things from industrial market perspective. *IEEE Access*. 2014;2:1660-1679.
2. Cooper D, Santesson S, Farrell S, Boeyen S, Housley R, Polk W. Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile. RFC 5280, RFC editor; 2008. <http://www.rfc-editor.org/rfc/rfc5280.txt>
3. IEEE standard for information technology –telecommunications and information exchange between systems –local and metropolitan area networks. Specific requirements–part 15.4: wireless medium access control (MAC) and physical layer (PHY) specifications for low-rate wireless personal area networks (LR-WPANs). <http://www.ieee802.org/15/pub/TG4.html>. 2003.
4. Ye W, Silva F, Heidemann J. Ultra-low duty cycle MAC with scheduled channel polling. In: Proceedings of the 4th International Conference on Embedded Networked Sensor Systems; 2006; Boulder, CO. <http://doi.acm.org/10.1145/1182807.1182839>
5. Buettner M, Yee GV, Anderson E, Han R. X-MAC: a short preamble MAC protocol for duty-cycled wireless sensor networks. In: Proceedings of the 4th International Conference on Embedded Networked Sensor Systems; 2006; Boulder, CO.
6. Dunkels A. *The ContikiMAC Radio Duty Cycling Protocol*. Technical Report No. T2011:13. Stockholm, Sweden: Swedish Institute of Computer Science; 2011. <http://dunkels.com/adam/dunkels11contikimac.pdf>
7. Bormann C, Hoffman P. Concise binary object representation (CBOR). RFC 7049, RFC Editor; 2013.
8. Mulligan G. The 6LoWPAN architecture. In: Proceedings of the 4th Workshop on Embedded Networked Sensors; 2007; Cork, Ireland. <http://doi.acm.org/10.1145/1278972.1278992>
9. Kushalnagar N, Montenegro G, Schumacher C. IPv6 over low-power wireless personal area networks (6LoWPANs): overview, assumptions, problem statement, and goals. RFC 4919, RFC Editor; 2007. <http://www.rfc-editor.org/rfc/rfc4919.txt>
10. Moharana SR, Jha VK, Satpathy A, Addya SK, Turuk AK, Majhi B. Secure key-distribution in IoT cloud networks. Paper presented at: 2017 Third International Conference on Sensing, Signal Processing and Security (ICSSS); 2017; Chennai, India.
11. Zhou J, Dong X, Cao Z, Vasilakos AV. Secure and privacy preserving protocol for cloud-based vehicular DTNs. *IEEE Trans Inf Forensics Secur*. 2015;10(6):1299-1314.
12. Zhou J, Cao Z, Dong X, Vasilakos AV. Security and privacy for cloud-based IoT: challenges. *IEEE Commun Mag*. 2017;55(1):26-33.
13. Stergiou C, Psannis KE, Kim B-G, Gupta B. Secure integration of IoT and cloud computing. *Future Gener Comput Syst*. 2018;78:964-975.
14. Soldatos J, Kefalakis N, Hauswirth M, et al. OpenIoT: open source internet-of-things in the cloud. In: *Interoperability and Open-Source Solutions for the Internet of Things*. Cham, Switzerland: Springer International Publishing; 2015:13-25.
15. Neto ALM, Souza ALF, Cunha I, et al. AoT: authentication and access control for the entire IoT device life-cycle. In: Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM; 2016; Stanford, CA. <http://doi.acm.org/10.1145/2994551.2994555>

16. Cirani S, Picone M, Gonizzi P, Veltri L, Ferrari G. IoT-OAS: an oauth-based authorization service architecture for secure services in IoT scenarios. *IEEE Sens J*. 2015;15(2):1224-1234.
17. Vuini M, Tourancheau B, Rousseau F, Duda A, Damon L, Guizzetti R. OSCAR: object security architecture for the Internet of Things. *Ad Hoc Netw*. 2015;32:3-16. Part of special issue: Internet of Things Security and Privacy: Design Methods and Optimization.
18. Wang T, Zhang G, Liu A, Bhuiyan MZA, Jin Q. A secure IoT service architecture with an efficient balance dynamics based on cloud and edge computing. *IEEE Internet Things J*. 2019.
19. Moosavi SR, Gia TN, Rahmani A-M, et al. Sea: a secure and efficient authentication and authorization architecture for IoT-based healthcare using smart gateways. *Procedia Comput Sci*. 2015;52:452-459. Part of special issue: The 6th International Conference on Ambient Systems, Networks and Technologies (ANT-2015), The 5th International Conference on Sustainable Energy Information Technology (SEIT-2015).
20. Kothmayr T, Schmitt C, Hu W, Brünig M, Carle G. DTLS based security and two-way authentication for the Internet of Things. *Ad Hoc Netw*. 2013;11(8):2710-2723.
21. Hummen R, Shafagh H, Raza S, Voig T, Wehrle K. Delegation-based authentication and authorization for the IP-based Internet of Things. Paper presented at: 2014 Eleventh Annual IEEE International Conference on Sensing, Communication, and Networking (SECON); 2014; Singapore.
22. Forsby F, Furuheid M, Papadimitratos P, Raza S. Lightweight X. 509 Digital Certificates for the Internet of Things. Paper presented at: Third International Conference, InterIoT 2017, and Fourth International Conference, SaSeIoT; 2017; Valencia, Spain.
23. Texas Instruments. MSP430F15x, MSP430F16x, MSP430F161x Mixed Signal Microcontroller Datasheet. <http://www.ti.com/lit/ds/symlink/msp430f1611.pdf>. 2011.
24. Atmel. Atmel 42129P SAM D20 Datasheet Complete. http://www.atmel.com/images/Atmel-42129-SAM-D20_Datasheet.pdf. 2016.
25. Atmel. SMART ARM-Based Flash MCU SAM3U Series Datasheet. <http://www.alldatasheet.com/datasheet-pdf/pdf/444352/ATMEL/SAM3U.html>. 2015.
26. Ye W, Heidemann J, Estrin D. An energy-efficient MAC protocol for wireless sensor networks. In: Proceedings of the Twenty-First Annual Joint Conference of the IEEE Computer and Communications Societies; 2002; New York, NY.
27. Dierks T, Rescorla E. The transport layer security (TLS) protocol version 1.2. RFC 5246, RFC Editor; 2008. <http://www.rfc-editor.org/rfc/rfc5246.txt>
28. Rescorla E, Modadugu N. Datagram transport layer security version 1.2. RFC 6347, RFC Editor; 2012. <http://www.rfc-editor.org/rfc/rfc6347.txt>
29. Housley R, Ford W, Polk T, Solo D. Internet X.509 public key infrastructure certificate and CRL profile. RFC 2459, RFC Editor; 1999. <http://www.rfc-editor.org/rfc/rfc2459.txt>
30. Winter T, Thubert P, Brandt A, et al. RPL: IPv6 routing protocol for low-power and lossy networks. RFC 6550, RFC Editor; 2012. <http://www.rfc-editor.org/rfc/rfc6550.txt>
31. Sam G. Introducing Azure IoT support for X.509 certificates. <https://azure.microsoft.com/en-us/blog/introducing-azure-iot-support-for-x-509-certificates/>. Accessed February 12, 2012.
32. Amazon. AWS IoT Core Documentation. <https://aws.amazon.com/documentation/iot/>. Accessed February 12, 2012.
33. Monsoon Solutions. Monsoon power monitor. <https://www.msoon.com/>. Accessed April 30, 2012.
34. Kovatsch M, Duquennoy S, Dunkels A. A low-power CoAP for Contiki. Paper presented at: 2011 IEEE Eighth International Conference on Mobile Ad-Hoc and Sensor Systems; 2011; Valencia, Spain.

How to cite this article: Kwon H, Ahn J, Ko J. LightCert: On designing a lighter certificate for resource-limited Internet-of-Things devices. *Trans Emerging Tel Tech*. 2019;30:e3740. <https://doi.org/10.1002/ett.3740>